

Examination Timetabling Problem

Nguyen Hoang Nam - 202416727

June 8, 2025

Abstract

The Examination Timetabling Problem (ETP) is a well-known NP-hard combinatorial optimization problem with significant practical relevance in educational institutions. It involves assigning a set of exams to available time slots and rooms while satisfying various hard and soft constraints. This study investigates five solution techniques: a Constraint Programming (CP) model, a Mixed Integer Programming (MIP) model, and two heuristic-based methods. Experimental results demonstrate that the heuristic-based approach consistently outperforms CP and MIP in terms of solution quality and computational efficiency. A comprehensive analysis of the strengths and limitations of each method is presented, along with discussions on possible improvements and future research directions. The findings contribute to the advancement of ETP-solving methodologies and provide actionable insights for practitioners aiming to optimize exam scheduling and related packing problems.

1 Introduction and Objectives

There are N classes $1, 2, \dots, N$ that need to be scheduled for the final exam. Each class must be assigned to a time-slot and a room. There are M rooms $1, 2, \dots, M$ that can be used for scheduling the exam. Each room i has capacity $c(i)$ (number of places in the room). Each day is divided into 4 slots $1, 2, 3, 4$. Each class i has number of students $d(i)$ ($i = 1, \dots, N$). Among N classes, there are K pairs of classes (i, j) in which class i and class j have the same students participating in the exam. It means that these 2 classes cannot be scheduled in the same time-slot. Compute the exam time-table such that the number of days used is minimal.

The data format

- First line: N and M
- Second lines: $d_1, d_2, \dots, d_N$
- Third lines: $c_1, c_2, \dots, c_M$
- Fourth line: K
- Last K lines: i and j (2 classes having a same student registered, these classes cannot be scheduled in the same slot)

Generated Data

The input size is denoted by $(N \times M)$.

- 15 tests:
 $(5 \times 2), (6 \times 2), (7 \times 3), (8 \times 3), (9 \times 4),$
 $(10 \times 3), (11 \times 4), (12 \times 4), (13 \times 4), (14 \times 4),$
 $(15 \times 4), (16 \times 4), (17 \times 4), (18 \times 5), (19 \times 5)$
- 10 tests:
 $(50 \times 10), (51 \times 10), (52 \times 11), (53 \times 11), (54 \times 11),$
 $(55 \times 11), (56 \times 12), (57 \times 12), (58 \times 12), (59 \times 13)$
- 10 tests:
 $(100 \times 10), (200 \times 10), (300 \times 10), (400 \times 10), (500 \times 10),$
 $(600 \times 11), (700 \times 11), (800 \times 11), (900 \times 10), (1000 \times 10)$
- 3 tests mainly for heuristic testing:
 $(1000 \times 20), (2000 \times 20), (3000 \times 20)$

Each test case is randomly generated as follows:

- Each class has a number of students between 20 and 70.
- Let S be the largest number of students in any class. Each room has a capacity between $S/2$ and $3S/2$.
- The number of pairs of classes that share at least one student is at most $N(N+1)/4$.

2 Modeling the Problem

In the model, let:

- N : the number of classes
- M : the number of rooms
- d_i : the number of students in class i
- c_j : the capacity of room j
- P : the set of conflict pairs (i.e., pairs of classes that share at least one student)

2.1 CP Model

2.1.1 Decision Variables

- $slot_i \in \{1, \dots, N\}$: time-slot assigned to class i
- $room_i \in \{1, \dots, M\}$: room assigned to class i

2.1.2 Constraints

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$d_i > c_j \Rightarrow \text{room}_i \neq j$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$\text{slot}_i \neq \text{slot}_j, \forall (i, j) \in P$$

- Only one class per room per time-slot:

$$\text{slot}_i = \text{slot}_j \Rightarrow \text{room}_i \neq \text{room}_j$$

2.1.3 Objective Function

Minimize the number of time-slots used:

$$\min \left(\max_{i=1}^N \text{slot}_i \right)$$

2.2 MIP Model

2.2.1 Decision Variables

- $x_{i,s,r} \in \{0, 1\}$: binary variable, equals 1 if class i is assigned to time-slot s and room r , otherwise 0
- $D \in [1, N]$: the maximum time-slot index used (i.e., latest slot with an exam scheduled)

2.2.2 Constraints

- A class can only be assigned to a room if its number of students does not exceed the room's capacity:

$$x_{i,s,r} = 0 \text{ if } c_j < d_i$$

- Each class is scheduled exactly once:

$$\sum_{s=1}^N \sum_{r=1}^M x_{i,s,r} = 1, \forall i \in \{1, \dots, N\}$$

- Classes that share at least one student cannot be scheduled in the same time-slot:

$$\sum_{\text{valid } r} (x_{i,s,r} + x_{j,s,r}) \leq 1, \forall (i, j) \in P, \forall s \in \{1, \dots, S\}$$

- Only one class per room per time-slot:

$$\sum_{i=1}^N x_{i,s,r} \leq 1, \forall s \in \{1, \dots, S\}, \forall r \in \{1, \dots, M\}$$

- Tracking the latest time-slot used:

$$\sum_{s=1}^N \sum_{r=1}^M (s+1) \cdot x_{i,s,r} \leq D, \forall i \in \{1, \dots, N\}$$

2.2.3 Objective Function

Minimize the number of time-slots used:

$$\min D$$

3 Heuristic Method

In this report, the term “cost” refers to the total number of days required to schedule all exams, which means that the max slot used.

3.1 Overview

This section presents two heuristic algorithms used to solve ETP, a **Local Search** and a **Iterated Local Search**. The **Iterated Local Search** is the enhanced version of the **Local Search** method, not only in solving faster, but also gives results which are as good as the original. However, the **Iterated Local Search** will take more memory than the another.

3.2 Sorting Input

Before applying either algorithm, we preprocess the input:

- **Room Fit Filtering:** For each class, only rooms with enough capacity are taken into consideration.
- **Class Order Randomization:** Classes are shuffled randomly before scheduling.

3.3 Process

- **Local Search:**
 - Starts from a valid initial solution generated randomly or via a Greedy approach.
 - Iteratively tries to move each class to an earlier valid time-slot and room.
 - A move is accepted if it reduces the maximum time-slot used, improving the overall schedule.
 - The process repeats until no further improving moves are found (local optimum is reached).
- **Iterated Local Search:**
 - Begins with an initial solution improved by Local Search.

- In each iteration, a small set of class assignments is randomly perturbed (i.e., removed and reassigned).
- Local Search is applied again to refine the perturbed solution.
- If the new solution is better (uses fewer time-slots), it becomes the current best.
- This process continues for a fixed number of iterations to escape local optima and explore broader solution space.

3.4 Process Demonstration

Local Search Algorithm

1. Start from a valid initial solution (generated by Greedy or Random assignment).
2. Build the current room usage and conflict tracking data.
3. For each class:
 - Try moving the class to an earlier valid time-slot and room where:
 - There is no conflict with other classes.
 - The room has enough capacity and is available.
 - If such a move improves the schedule, apply it.
4. Repeat until no further improvements can be made.

Iterated Local Search Algorithm

1. Generate an initial solution (using random assignment and local search).
2. Save this as the current best solution.
3. Repeat for a fixed number of iterations:
 - Start from the current best solution.
 - **Perturbation:** randomly select and remove a small number of class assignments, then reassign them randomly to valid time-slots and rooms.
 - Apply Local Search to improve the perturbed solution.
 - If the new solution uses fewer days:
 - Update it as the new best solution.

3.5 Pseudo-code

The following pseudo-codes illustrate the implementation of the Local Search, and Iterated Local Search algorithms used in the proposed approach to solve the Exam Timetabling Problem (ETP).

Each algorithm is designed to explore the solution space effectively, and the pseudo-codes serve as a practical guideline for implementation and future optimization efforts.

Algorithm 1 Local Search for Course Timetabling

Input: An initial solution S

Output: A locally optimized solution S

```
1  $improved \leftarrow \mathbf{true}$ 
   while  $improved$  do
2    $improved \leftarrow \mathbf{false}$ 
     foreach  $class\ c\ in\ S$  do
3     earlier slot  $s$  foreach  $room\ r$  do
4       if assigning  $c$  to  $(s, r)$  is valid and reduces  $cost(S)$  then
5         Apply the assignment of  $c$  to  $(s, r)$     $improved \leftarrow \mathbf{true}$ 
         break all loops
6 return  $S$ 
```



Algorithm 2 Iterated Local Search algorithm

Input: Maximum number of iterations N **Output:** Best solution S^*

```
7  $S \leftarrow \text{GenerateInitialSolution}()$ 
    $S \leftarrow \text{LocalSearch}(S)$ 
    $S^* \leftarrow S$ 

8 for  $i \leftarrow 1$  to  $N$  do
9    $S' \leftarrow \text{Perturb}(S^*)$ 
    $S' \leftarrow \text{LocalSearch}(S')$ 
   if  $\text{cost}(S') < \text{cost}(S^*)$  then
10     $S^* \leftarrow S'$ 
11 return  $S^*$ 
```

Function Perturb(S)

```
1  $S' \leftarrow \text{Copy}(S)$ 
2 Select  $k$  random classes from  $S'$ 
3 foreach selected class  $c$  do
4   Remove current assignment of  $c$ 
5   Choose a random valid time slot  $s$  and room  $r$ 
6   if assignment  $(s, r)$  is valid then
7     Assign  $c$  to  $(s, r)$ 
8   else
9     Reassign  $c$  to original slot and room
10 return  $S'$ 
```

4 Result and Analysis

4.1 Result

4.1.1 CP and MIP model Solver

Exact Solution

Input sizes		CP		MIP	
N	M	f	t(ms)	f	t(ms)
5	2	5	7	5	35
6	2	3	12	3	85
7	3	3	18	3	162
8	3	3	17	3	278
9	4	4	27	4	123
10	3	4	26	4	164
11	4	6	31	6	423
12	4	4	43	4	577
13	4	5	142	5	902
14	4	8	93	8	1334
15	4	9	72	9	28636
16	4	8	173	8	21724
17	4	10	188	F	F
18	5	7	189	F	F
19	5	8	13841	F	F
50	10	5	7345	F	F
51	10	F	F	F	F
52	11	F	F	F	F
53	11	F	F	F	F

Table 1: Results only exact solution of CP solver and MIP solver

All Solution

As a result of the large size and intricate nature of the optimization problems under consideration, we were only able to obtain feasible solutions for test instances with input size smaller than or equal to (53×10) for the CP solver, and (16×4) for the MIP solver, within a time limit of 300 seconds. This indicates that the optimization problems are highly complex and computationally demanding, requiring significant computational resources to solve. To ensure the reliability of our findings, we conducted ten runs of each solver and computed the average results, thereby mitigating the potential impact of random variations and ensuring the robustness of our numerical experiments.

Input sizes		CP				MIP			
N	M	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$
5	2	5	5	5	7	5	5	5	35
6	2	3	3	3	12	3	3	3	85
7	3	3	3	3	18	3	3	3	162
8	3	3	3	3	17	3	3	3	278
9	4	4	4	4	27	4	4	4	123
10	3	4	4	4	26	4	4	4	164
11	4	6	6	6	31	6	6	6	423
12	4	4	4	4	43	4	4	4	577
13	4	5	5	5	142	5	5	5	902
14	4	8	8	8	93	8	8	8	1334
15	4	9	9	9	72	9	9	9	28636
16	4	8	8	8	173	8	8	8	21724
17	4	10	10	10	188	10	10	10	301973
18	5	7	7	7	189	7	7	7	303064
19	5	8	8	8	13841	8	8	8	302770
50	10	5	5	5	7345	5	5	5	321161
51	10	7	8	7.33	300154	9	9	9	325693
52	11	8	9	8.33	300164	15	15	15	325189
53	11	14	14	14	300537	15	15	15	315767
54	11	9	11	10	300218	9	11	10.33	316766
55	11	14	14	14	300202	15	15	15	314033
56	12	9	10	9.67	300191	13	15	14	326837
57	12	9	12	11	300184	18	18	18	337534
58	12	9	11	10.33	300222	23	27	25.33	339576
59	13	9	9	9	300262	17	17	17	342626
100	10	18	18	18	300696	F	F	F	F
200	10	183	210	195.33	302356	F	F	F	F
300	10	F	F	F	F	F	F	F	F
400	10	F	F	F	F	F	F	F	F
500	10	F	F	F	F	F	F	F	F
600	11	F	F	F	F	F	F	F	F
700	11	F	F	F	F	F	F	F	F
800	11	F	F	F	F	F	F	F	F
900	10	F	F	F	F	F	F	F	F
1000	10	F	F	F	F	F	F	F	F
1000	20	F	F	F	F	F	F	F	F
2000	20	F	F	F	F	F	F	F	F
3000	20	F	F	F	F	F	F	F	F

Table 2: Average results of CP solver and MIP solver

4.1.2 Heuristic Solver

Input sizes		Local Search				Iterated Local Search			
N	M	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$	f_{min}	f_{max}	f_{avg}	$t_{avg}(ms)$
5	2	5	5	5	157	5	5	5	202
6	2	3	3	3	157	4	4	4	209
7	3	3	3	3	158	4	4	4	219
8	3	3	3	3	160	4	4	4	228
9	4	4	4	4	162	5	5	5	248
10	3	4	4	4	159	4	4	4	264
11	4	6	7	6.3	163	7	7	7	277
12	4	4	4	4	164	6	6	6	277
13	4	5	6	5.3	164	8	8	8	293
14	4	8	8	8	165	9	9	9	311
15	4	9	9	9	164	11	11	11	311
16	4	8	8	8	164	9	9	9	312
17	4	10	10	10	164	11	11	11	319
18	5	7	10	7.3	173	9	9	9	329
19	5	8	9	8.5	168	10	10	10	337
50	10	8	10	9	199	11	11	11	452
51	10	9	10	9.2	199	11	11	11	450
52	11	9	10	9.9	207	11	11	11	449
53	11	14	14	14	208	15	15	15	478
54	11	11	12	11.9	205	13	13	13	472
55	11	16	17	16.5	210	17	17	17	553
56	12	11	12	11.3	207	12	12	12	463
57	12	11	12	11.2	212	12	12	12	460
58	12	11	12	11.8	214	13	13	13	479
59	13	11	12	11.6	229	12	12	12	466
100	10	20	21	20.9	310	23	23	23	573
200	10	33	34	33.2	896	34	36	35	912
300	10	49	51	49.8	2600	52	54	52.8	1500
400	10	64	66	65.1	5515	66	70	67.7	2688
500	10	71	72	71.3	10449	73	76	74.3	3385
600	11	91	92	91.7	18215	93	96	94.6	6077
700	11	91	92	91.6	39295	93	96	94.3	5829
800	11	159	160	159.6	43907	161	162	161.2	29096
900	10	127	128	127.5	55770	130	133	131.3	15203
1000	10	183	184	183.2	74038	185	188	185.9	40017
1000	20	122	123	122.4	228972	121	128	125.6	13436
2000	20	F	F	F	F	180	183	181.2	62798
3000	20	F	F	F	F	178	180	179.5	232967

Table 3: Average results of Heuristic solver

4.2 Analysis

4.2.1 Exact Solution

Our evaluation of the accuracy of each algorithm revealed that CP and MIP were capable of generating exact solutions for a subset of test cases:

- CP was able to produce exact solutions for tests with sizes: (5×2) , (6×2) , (7×3) , (8×3) , (9×4) , (10×3) , (11×4) , (12×4) , (13×4) , (14×4) , (15×4) , (16×4) , (17×4) , (18×5) , (19×5) , (50×10) , (51×10) , (52×11) , (53×11)
- MIP was only able to produce exact solutions for tests with sizes: (5×2) , (6×2) , (7×3) , (8×3) , (9×4) , (10×3) , (11×4) , (12×4) , (13×4) , (14×4) , (15×4) , (16×4)

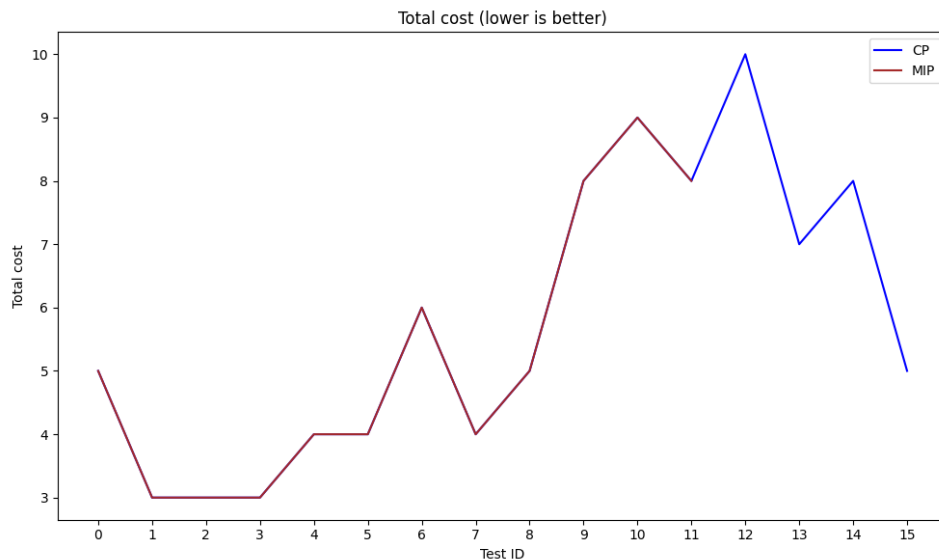


Figure 1: Results compare only exact solutions

4.2.2 All Solution

Our analysis of the solutions generated by five distinct algorithms revealed the following:

- CP is unable to handle datasets larger than (200×10) .
- MIP is unable to handle datasets larger than (59×13) .
- LS fails to solve two test cases: (2000×20) and (3000×20)
- ILS is capable of handling all test cases.

4.2.3 Total cost

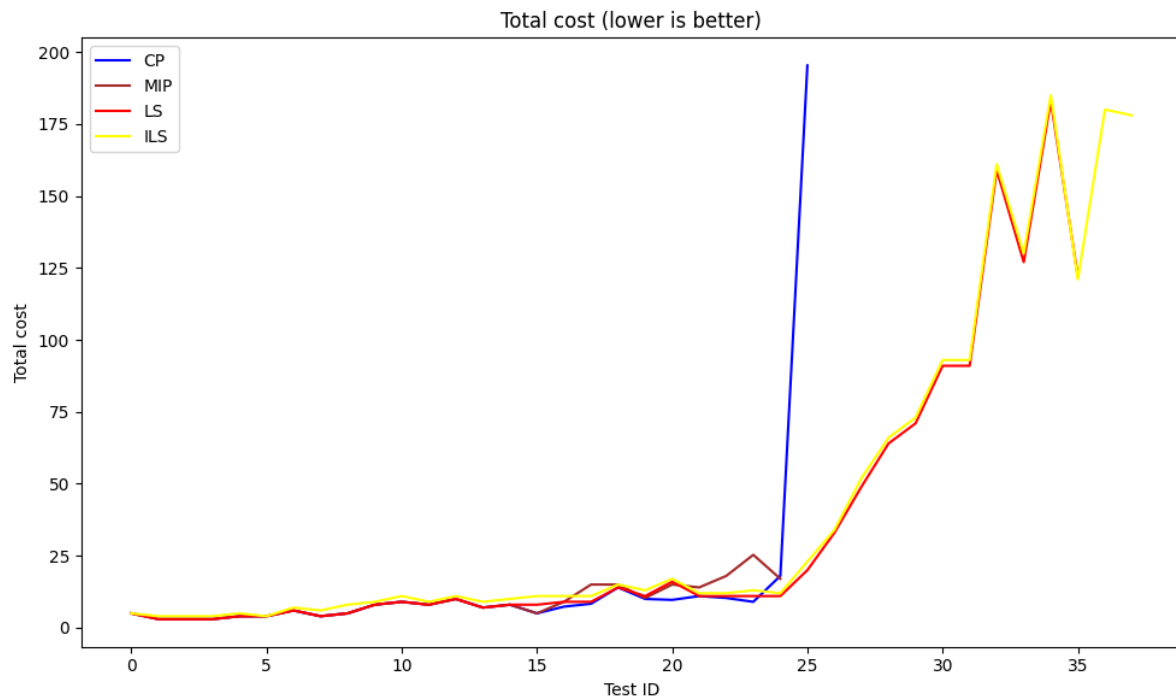


Figure 2: Total cost comparison between CP, MIP, LS and ILS heuristics

- Overall, in the first 20 test case, all method generate nearly equivalent results. However from test ID 25 onward, when the size is larger than (59 x 13), LS and ILS consistently yield the lowest total costs, both significantly outperform CP and MIP.

In conclusion, ILS offers the best performance across all test sizes, followed closely by LS. These heuristic methods are well-suited for larger and more complex scheduling problems, where exact methods like CP and MIP struggle.

4.2.4 Runtime

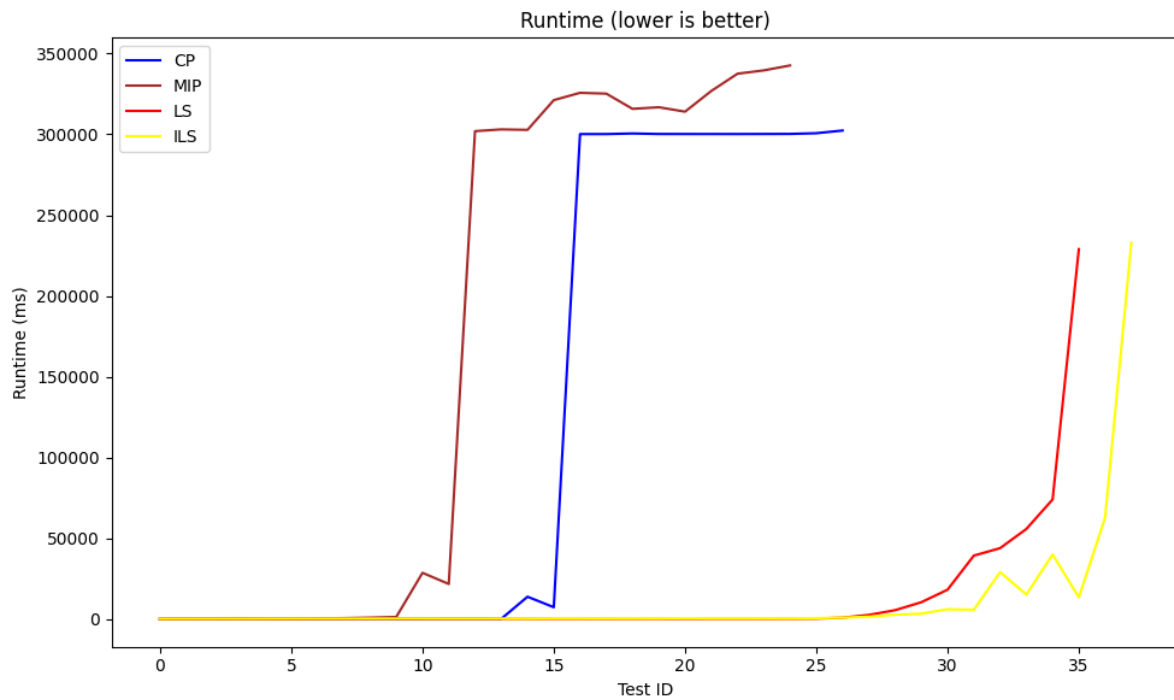


Figure 3: Runtime comparison between LS and ILS heuristics

- Overall, this graph demonstrates that ILS can handle input best, followed by LS, and it is important to note that they outperform CP and MIP methods.

From the graph, MIP reaches the time limit of 300 seconds for all tests with size greater than or equal (17×4). While CP reaches the time limit of 300 seconds for all tests with size greater than or equal (51×10). Moving to LS and ILS, it is important to note that ILS can handle input better than LS, as we can see in the final test cases (test case 28th onward), where size greater than (300×10)

4.2.5 The analysis of LS and ILS

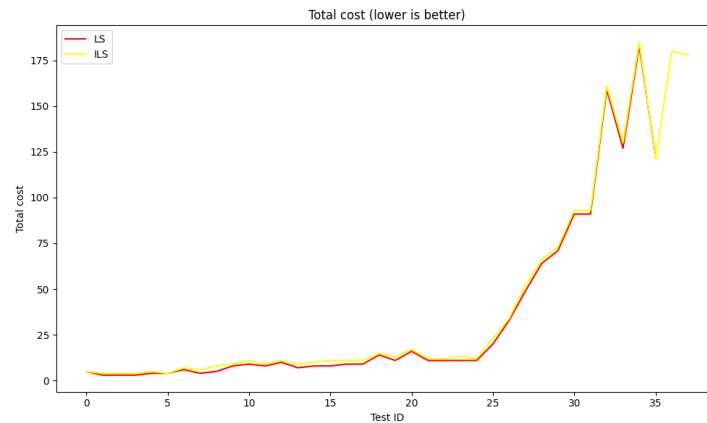


Figure 4: Total cost comparison between LS and ILS heuristics

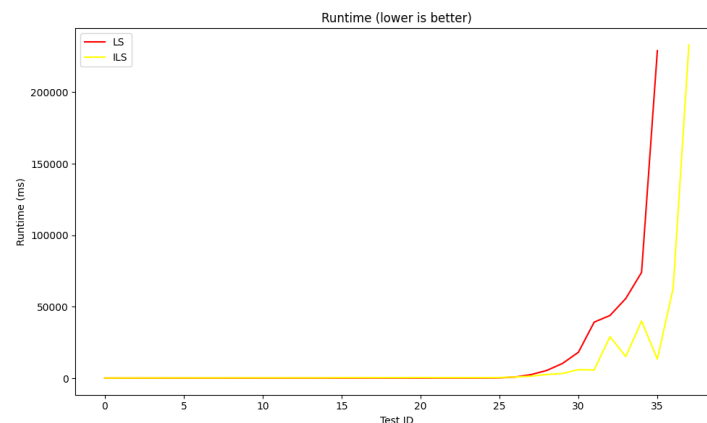


Figure 5: Runtime comparison between LS and ILS heuristics

- Overall, Iterated Local Search (ILS) is the enhanced version of Local Search (LS), aiming to solve large test cases, particularly from test 30 (size of: 500 x 10) onward. For example, in test 34 (size of: 900 x 10), the running time of LS method is over 200000 ms, while that of ILS is far lower (below 50000 ms). It is also important to note that, from test case 37 and 38 with the size of input larger than (2000 x 20), ILS is capable of solving them, while LS meets struggle.

⇒ ILS not only gives us results as good as LS, but also outperforms it in terms of runtime, demonstrating greater efficiency, especially for large test cases.

5 Conclusion

In conclusion, our evaluation demonstrates that the heuristic algorithms are both accurate and efficient, with the ILS (the enhanced version of LS) algorithm standing out for its combination of fast runtimes and strong solution quality. These characteristics make heuristic methods particularly well-suited for practical applications that demand both speed and accuracy. While the CP solver remains a viable option, especially for smaller datasets, the MIP solver consistently underperforms and is therefore not recommended.

6 Limitations and Future research

Our analysis also revealed some limitations and potential avenues for future research. Firstly, our study only considered a specific set of test cases, and the impact of varying problem sizes and parameter settings was not investigated. Therefore, future research could focus on evaluating the performance of the algorithms on a wider range of test cases and exploring the impact of different parameter settings. Additionally, future research could focus on optimizing the performance of the Heuristic algorithm while maintaining its accuracy and efficiency, or explore the potential for hybrid algorithms that combine elements of the CP, MIP, and Heuristic models to achieve even better performance. Furthermore, our study only focused on the theoretical aspects of the optimization problem and did not consider the practical implications of implementing the algorithms in real-world scenarios. Thus, future research could investigate the potential for implementing these algorithms in various domains and evaluating their performance in practical settings.



7 Visualization

In terms of visualization, we generated figures to display the results of the CP, MIP, LS, and ILS solvers. These figures allow for an intuitive comparison of how each algorithm assigns classes to time slots and rooms. The slot colors are consistent across all figures for clarity.

Sample Test Case

```
10 3
72 77 71 71 53 45 53 53 66 70
79 53 70
16
1 2
1 3
1 8
1 10
2 5
2 9
3 6
3 9
4 10
5 8
5 10
7 8
7 9
7 10
8 9
9 10
```

Slot 1	Slot 2
Slot 3	Slot 4

Class	Room 1	Room 2	Room 3
Class 1	Slot 1		
Class 2	Slot 4		
Class 3	Slot 3		
Class 4	Slot 2		
Class 5		Slot 1	
Class 6		Slot 2	
Class 7			Slot 4
Class 8			Slot 2
Class 9			Slot 1
Class 10			Slot 3

(a) CP solver

Class	Room 1	Room 2	Room 3
Class 1	Slot 2		
Class 2	Slot 4		
Class 3	Slot 1		
Class 4	Slot 3		
Class 5		Slot 3	
Class 6		Slot 2	
Class 7			Slot 1
Class 8			Slot 2
Class 9			Slot 3
Class 10			Slot 4

(b) MIP solver

Class	Room 1	Room 2	Room 3
Class 1	Slot 1		
Class 2	Slot 2		
Class 3	Slot 3		
Class 4	Slot 4		
Class 5		Slot 1	
Class 6			Slot 1
Class 7		Slot 2	
Class 8		Slot 3	
Class 9			Slot 4
Class 10			Slot 3

(c) LS solver

Class	Room 1	Room 2	Room 3
Class 1	Slot 1		
Class 2	Slot 2		
Class 3	Slot 3		
Class 4	Slot 4		
Class 5		Slot 1	
Class 6			Slot 1
Class 7		Slot 2	
Class 8		Slot 3	
Class 9			Slot 4
Class 10			Slot 3

(d) ILS solver

Figure 6: Result visualization for test size 10×3

References

- [1] E. S. K. Siew, S. N. Sze, S. L. Goh, G. Kendall, N. R. Sabar, and S. Abdullah, *A Survey of Solution Methodologies for Exam Timetabling Problems*, IEEE Access, 2024.
- [2] Pillay, N., Qu, R., *Examination Timetabling Problems. In: Hyper-Heuristics: Theory and Applications. Natural Computing Series*, Springer, Cham, 2018.
- [3] Lourenço, O. C., Martin, M., Stützle, T. (2003), Iterated Local Search in Handbook of Metaheuristics, Springer.